



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 04- Part 2

Introduction to Classes and Objects





THIS WEEK'S AGENDA

- 01. Revisiting OOP Motivation
- 02. Classes and Objects
- 03. Principles of OOP





MOTIVATION FOR OOP





MOTIVATION FOR OOP

To understand the motivation behind OOP, let's consider how we have been dealing with strings in C++ so far.

Creating strings in C++ using arrays (c-strings) is a tedious task and requires a lot of manual work such as:

- The user has to keep track of the length of the string.
- The user has to write custom functions in order to manipulate and work with `char*` (c-strings), such as a function to concatenate two c-strings.

However, the C++ standard provides us with the string class (`std::string`). Here string is basically a class within the `std` namespace.





CLASSES AND OBJECTS





CLASSES IN C++

- A class in Object Oriented Programming is a collection of
 - **data**, and
 - various **operations** performed on that data.
- It consists of variables of different data types or other classes, as well as functions on these variables.

```
3  class Multiplier {  
4  private:  
5      int a;  
6      int b;  
7  public:  
8      Multiplier(int x, int y) {  
9          a = x;  
10         b = y;  
11     }  
12     int mult() {  
13         return a * b;  
14     }  
15 };
```





OBJECTS IN C++

- A class in Object Oriented Programming can be thought of as a data type.
- However, classes are more advanced than primitive data types.
- This is because classes contain both data and functions relating to it.

Conversely, an object is an **instance** of a particular class.

- In terms of primitive data types, an object can be thought of as similar to a variable that may hold some value.





OBJECTS IN C++

An object is an **instance** of a particular class.
Here **m1** is an **object** of the **Multiplier** class.

```
18  int main() {  
19      // Create an object of Multiplier class  
20      Multiplier m1(2, 3);  
21      // Call the mult method and print the result  
22      std::cout << "mult(2, 3) = " << m1.mult() << std::endl;  
23      return 0;  
24  }
```





PRINCIPLES OF OOP





ABSTRACTION

- Abstraction helps in simplifying the complex working of a class or a method by hiding the unnecessary details of it and only highlighting the very important ones.
- Think of **how** a car is driven. You need to only know that pressing the accelerator takes it forward while pressing the break stops it.
- You need not know the inner workings in order to use it.
- Similarly, you need not know the inner working of say, a particular method of a class. You only need to know that it can be accessed from an object of that class.





ENCAPSULATION

- Encapsulation is the practice of keeping the data (variables) and the methods that manipulate it together in a class, while restricting direct access to some of the object's components.
- This means that the class which owns the data and the methods decides the degree to which they can be accessed and changed. This is done with the help of the **private** and **public** keywords.
- In other words, it uses **access specifiers** to protect the internal state of our objects from outside entities.





INHERITENCE

- Inheritance allows a new class to acquire and use the properties and methods of a pre-existing class. This promotes code re-usability.
- Via inheritance, the child class can use the features of the parent class. Moreover, inheritance conveys an “is-a” relationship, which means that the derived class is a special version of the base class.
 - **Base Class** - The main class in the inheritance from where the derived class uses the properties and features. It is also called the **parent class**.
 - **Derived Class** - The class that uses the features of the base/parent class. It is also called the **child class**.





INHERITENCE

```
1  #include <iostream>
2
3  // Parent class
4  class Vehicle {
5      public:
6      void start() {
7          std::cout << "Vehicle started" << std::endl;
8      }
9  };
10
11 class Car: public Vehicle {
12     public:
13     void start() {
14         std::cout << "Car started" << std::endl;
15     }
16 };
```





POLYMORPHISM

- Polymorphism means “many forms” (in literal terms).
- In terms of OOP, it means allowing objects of different classes to be treated as instances of a common superclass, and methods to behave differently based on the calling object. There are two primary types of polymorphism:
 - **Compile-time(Method/Function Overloading)** - Same method/function name with different parameters (number and type), or different order of parameters.
 - **Run-time (Method Overriding)** - A subclass provides specific implementations of a superclass method.





SUMMARY





SUMMARY

- In C++ all dynamically allocated memory must be explicitly deallocated or freed in order to avoid memory leaks.
- A dangling pointer is a pointer that refers to a variable or a data element that has been deleted and deallocated. Dangling pointers must be set to nullptr.
- Function overloading refers to creating functions of the same name but different function signature in terms of number of parameters, their types or order. Overloaded functions need to have the same name.
- Classes in C++ are built upon the following four pillars: Abstraction, Encapsulation, Inheritance and Polymorphism.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 06





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

